

Atelier — Planètes (fiches)

À ouvrir après 1-COURS.pdf du dossier projets/01-planetes/.

1–3 périodes. Quatre fiches. Rien ne tourne.

Le dossier depart/ de cet exercice contient les fichiers de départ. Vous n'avez pas à ouvrir un terminal.

Où l'on va

Vous allez fabriquer Terre, puis lui poser une question. Ensuite, Saturne permettra de comparer deux objets. Quand ces deux gestes fonctionnent, vous créez les quatre fiches du catalogue.

À la fin, votre écran ressemble à ceci. Les cartes viennent des objets ; aucune planète ne tourne.

CABINET DES FICHES
Les quatre
La méthode

SAS · PLANÈTES · FICHES

Quatre fiches, aucun tour de manège.

Mercure, Terre, Saturne, Neptune. Quatre objets `Planete`. Nom, distance, gazeuse oui ou non. Rien ne tourne.

Le catalogue

Les cartes viennent du script, pas de quatre blocs copiés.

Mercure DISTANCE 0.39 UA SATELLITES 0 TYPE rocheuse GAZEUSE : NON	Terre DISTANCE 1 UA SATELLITES 1 TYPE rocheuse GAZEUSE : NON	Saturne DISTANCE 9.5 UA SATELLITES 146 TYPE gazeuse GAZEUSE : OUI	Neptune DISTANCE 30 UA SATELLITES 16 TYPE gazeuse GAZEUSE : OUI
---	--	---	---

Ce que l'objet sait

Saturne.plusLoinQue(Terre) répond oui. La comparaison vit dans la classe.

Corriolé · classe Planete · zéro canvas.

Sur un téléphone de 390 pixels de large, les cartes s'empilent et aucun mot n'est coupé.

CABINET DES FICHES

[Les quatre](#) [La méthode](#)

SAS · PLANÈTES · FICHES

Quatre fiches, aucun tour de manège.

Mercure, Terre, Saturne, Neptune. Quatre objets `Planete`. Nom, distance, gazeuse oui ou non. Rien ne tourne.

Le catalogue

Les cartes viennent du script, pas de quatre blocs copiés.

Mercure

DISTANCE

0.39 UA

SATELLITES

0

TYPE

rocheuse

GAZEUSE : NON

Terre

DISTANCE

1 UA

Vous créez exactement ceci

```
sas/  
├─ planetes/  
│  └─ index.html  
│  └─ css/  
│      └─ style.css  
│  └─ js/  
│      └─ planete.js
```

Le départ contient déjà le HTML et le CSS. Votre travail se fait dans `js/planete.js` : vous complétez deux méthodes, vous créez les objets, puis vous les affichez.

Étape 1 — Ouvrir le départ

La page est déjà construite et mise en forme. Les cartes sont encore vides parce que le script n'a fabriqué aucun objet : c'est l'état normal du départ.

Placez le dossier donné dans `sas/planetes/`, puis ouvrez `sas/planetes/index.html` par un double-clic. Vous devez voir le titre « Quatre fiches, aucun tour de manège », un fond clair et une zone de catalogue vide.

- ☐ L'arbre correspond exactement au dessin ci-dessus.
- ☐ Le titre et la navigation sont visibles.
- ☐ Le catalogue est vide.
- ☐ La console du navigateur ne montre pas d'erreur rouge.

Si la page n'est pas mise en forme, vérifiez que `index.html` charge `css/style.css`. Vous n'avez à recopier ni le HTML ni le CSS depuis ce PDF.

Étape 2 — Fabriquer un seul objet : Terre

Une fiche HTML recopiée quatre fois oblige à maintenir quatre blocs séparés. Une **classe** évite cette répétition : c'est le moule. `new Planete(...)` utilise ce moule pour produire **un objet réel**, comme un moule à billets produit un billet réel.

Ouvrez `js/planete.js`. Le constructeur du départ reçoit quatre valeurs et les range sur l'objet avec `this : nom, distanceUa, satellites` et `type`.

Sous la classe, fabriquez seulement la Terre :

```
const terre = new Planete("Terre", 1, 1, "rocheuse");
```

Ouvrez les outils de développement du navigateur, puis la console. Tapez `terre.nom`, puis `terre.distanceUa`. Vous devez lire "Terre", puis 1. `Planete` est le moule ; `terre` désigne cet objet précis.

Étape 3 — Terre répond elle-même

Vous venez de lire le type de Terre. Si chaque endroit de la page compare lui-même ce texte, la règle sera répétée. Une **méthode** place la question sur l'objet qui connaît déjà son type.

Dans `estGazeuse()`, remplacez le commentaire `/* à vous : une ligne */`. La règle nommée **planète gazeuse** est :

```
estGazeuse() {  
  /* à vous : une ligne */  
}
```

```
retourner vrai quand le type vaut "gazeuse"
```

Rechargez la page. Dans la console, tapez :

```
terre.estGazeuse();
```

Vous devez obtenir `false`. Vous ne demandez pas à la page de relire `terre.type` : vous parlez à l'objet, et il répond.

Étape 4 — Deux objets se comparent

Une planète peut aussi répondre à une question qui concerne un autre objet. `plusLoinQue(autre)` reçoit donc un objet `Planete` et compare les deux distances en UA.

Dans le squelette de `plusLoinQue(autre)`, écrivez la règle nommée **plus loin qu'une autre planète** :

```
plusLoinQue(autre) {
  /* à vous : une ligne */
}
```

```
retourner vrai quand la distance de cet objet est supérieure à celle de autre
```

Ajoutez Saturne sous Terre :

```
const saturne = new Planete("Saturne", 9.5, 146, "gazeuse");
```

Rechargez, puis testez dans la console :

```
saturne.plusLoinQue(terre);
terre.plusLoinQue(saturne);
```

Vous devez obtenir `true`, puis `false`. La méthode compare **deux objets** : `this` est celui placé avant le point, et `autre` est celui entre parenthèses.

Étape 5 — Quatre objets, une page mince

Terre sait répondre ; Saturne sait se comparer à Terre. Vous pouvez maintenant utiliser quatre fois le même moule. Remplacez vos deux constantes par ce catalogue :

```
const catalogue = [
  new Planete("Mercure", 0.39, 0, "rocheuse"),
  new Planete("Terre", 1, 1, "rocheuse"),
  new Planete("Saturne", 9.5, 146, "gazeuse"),
  new Planete("Neptune", 30, 16, "gazeuse"),
];
```

La page doit seulement parcourir ces objets et leur poser la question. Sous le commentaire `/* étape 5 : affichage */` du départ, ajoutez ce bloc :

```
const liste = document.getElementById("liste");
```

```

catalogue.forEach((planete) => {
  const gazeuse = planete.estGazeuse();
  const article = document.createElement("article");
  article.className = "fiche";
  article.innerHTML = `
<h3>${planete.nom}</h3>
<dl>
<dt>Distance</dt><dd>${planete.distanceUa} UA</dd>
<dt>Satellites</dt><dd>${planete.satellites}</dd>
<dt>Type</dt><dd>${planete.type}</dd>
</dl>
<p class="badge ${gazeuse ? "badge--oui" : "badge--non"}">
Gazeuse : ${gazeuse ? "oui" : "non"}
</p>`;
  liste.appendChild(article);
});

const terre = catalogue[1];
const saturne = catalogue[2];
document.getElementById("comparaison").textContent =
saturne.plusLoinQue(terre)
? "Saturne.plusLoinQue(Terre) répond oui."
: "Vérifiez les distances des deux objets.";

```

Rechargez. Mercure et Terre doivent répondre « Gazeuse : non » ; Saturne et Neptune doivent répondre « Gazeuse : oui ». La boucle demande `estGazeuse()` aux objets : elle ne réécrit pas la règle `type === "gazeuse"`.

Réussi si

- ☐ Vous savez montrer Terre seule et expliquer la différence entre `Planete` et `terre`.
- ☐ `terre.estGazeuse()` répond `false`.
- ☐ `saturne.plusLoinQue(terre)` répond `true`.
- ☐ Les quatre objets utilisent les données indiquées.
- ☐ La page demande `estGazeuse()` aux objets sans recalculer la règle.
- ☐ La page se lit à 1440 et à 390 pixels de large.
- ☐ Rien ne tourne et aucun canvas n'a été ajouté.

AVEC L'IA

Vous avez écrit les deux méthodes vous-même. Montrez seulement votre boucle à une IA et demandez : « Expliquez en quatre phrases qui décide si une planète est gazeuse. » Vérifiez que sa réponse désigne `estGazeuse()` et non une comparaison recopiée dans la page.

POUR LES RAPIDES

Depuis la console, demandez `neptune.plusLoinQue(saturne)`, puis prédisez la réponse inverse avant de la tester. Ajoutez ensuite une méthode `resume()` qui retourne le nom et la distance de l'objet, sans créer une cinquième planète.

Livrable

Rendez le dossier `sas/planetes/` avec ses trois fichiers. La page doit s'ouvrir par double-clic. À l'oral, vous montrez un `new Planete(...)`, puis vous expliquez pourquoi `estGazeuse()` et `plusLoinQue(autre)` vivent dans la classe.